STOCKPORT COUNTY FC
TECHNICAL IMPLEMENTATION GUIDE

Code-Level Build Specification for LLM-Assisted Development

Companion to: Stockport County FC Recruitment Model Specification v2.0

March 2026 | Confidential

How to Use This Document

This document translates the Stockport County Recruitment Model Specification v2.0 into concrete coding instructions. Each section maps directly to a section of the spec and tells an LLM (or a developer) exactly what to build: which files to create, which functions to write, which database tables to define, and in what order.

Build order matters. Components are numbered in dependency order. Do not skip ahead. Each component depends on the ones before it.

Technology stack:

- **Language:** Python 3.11+
- **Database:** PostgreSQL 15+
- **ORM:** SQLAlchemy 2.0 (for schema definition and queries)
- **Data processing:** pandas, numpy
- **ML:** scikit-learn (gradient boosting, logistic regression, k-means), statsmodels (hierarchical regression)
- **API calls:** requests
- **Web scraping:** BeautifulSoup4 (for FBref)
- **Output:** Jinja2 templates for PDF reports, or direct HTML
- **Scheduling:** cron or APScheduler

Project root: `~/stockport_model/`

> **Prompt Pattern for LLM Code Generation**
> For each component below, feed the LLM: (1) this component's specification, (2) the database schema it depends on, (3) any upstream component outputs it consumes, and (4) the test criteria. Ask the LLM to generate the file, then review before moving to the next component. Do NOT ask the LLM to generate the entire codebase at once.

Component 0: Project Structure

Create this directory structure before building any component:

Path	Purpose
stockport_model/	Project root
config/	Configuration files: API keys, league lists, role templates
db/	Database schema definitions, migrations, connection
ingestion/	Data ingestion scripts (API-Football, FBref, Transfermarkt)
features/	Feature engineering: per-90, rolling, adjustments
models/	ML model training, prediction, validation
gates/	Hard gate filtering engine
scoring/	Composite scoring, archetype weighting, VAR
outputs/	Report generation, shortlist cards, comparison sheets
governance/	Override logging, prediction tracking, audit
tests/	Unit tests and integration tests per component
data/	Local data cache (CSV backups, FBref scrapes)
templates/	Jinja2 templates for PDF/HTML report outputs

Key config files to create first:

- `config/settings.py` — API keys, database connection string, file paths, constants
- `config/leagues.json` — Tracked leagues with league_id, name, country, tier, polling priority
- `config/role_templates.json` — Role template definitions with metric weights (Section 8 of spec). Versioned.
- `config/archetype_weights.json` — Scoring layer weights per archetype (Section 5.3 of spec)
- `config/hard_gates.json` — Gate definitions with class (absolute/caution/exception) and thresholds
- `config/age_curves.json` — Position-specific age curves from Section 6.1

Component 1: Database Schema

File: `db/schema.py`

Define all tables using SQLAlchemy declarative base. Run migration to create tables in PostgreSQL. Every table below is required before any ingestion begins.

players

Primary key: `player_id (PK, int)`

Columns: player_name (text), nationality (text), birth_date (date), height_cm (int, nullable), weight_kg (int, nullable), photo_url (text, nullable), current_team (text, nullable), current_league_id (int, nullable)

Notes: Biographical data. Source: API-Football /players. Updated weekly.

match_performances

Primary key: `fixture_id + player_id (composite PK)`

Columns: league_id, season, date, home_team, away_team, team, is_home (bool), referee, minutes, position, rating, is_substitute (bool), is_captain (bool), goals_scored, goals_conceded, assists, saves, shots_total, shots_on_target, passes_total, passes_key, pass_accuracy, tackles_total, tackles_blocks, tackles_interceptions, duels_total, duels_won, dribbles_attempts, dribbles_success, dribbles_past, fouls_committed, fouls_drawn, yellow_cards, red_cards, pen_won, pen_committed, pen_scored, pen_missed, pen_saved, offsides

Notes: Core per-match stats. Source: API-Football /fixtures/players. ALL fields captured. No minimum minutes filter at ingestion. Null stored as null, never 0.

fixtures

Primary key: `fixture_id (PK, int)`

Columns: league_id, season, date, home_team, away_team, home_score, away_score, referee, status

Notes: Match metadata. Source: API-Football /fixtures.

fixture_team_stats

Primary key: `fixture_id + team_name (composite PK)`

Columns: possession, total_shots, shots_on_target, corners, fouls, expected_goals (nullable), passes_total, passes_accuracy

Notes: Team-level match context. Source: API-Football /fixtures/statistics.

match_events

Primary key: `id (PK, auto)`

Columns: fixture_id, time_elapsed, time_extra, event_type, event_detail, player_id, player_name, assist_player_id, assist_player_name, team, comments

Notes: Goals, cards, subs, VAR. Source: API-Football /fixtures/events.

lineups

Primary key: `fixture_id + player_id (composite PK)`

Columns: team, is_starter (bool), position_label, grid_position, shirt_number, formation, coach_name, coach_id

Notes: Starting XI + bench. Source: API-Football /fixtures/lineups.

standings_snapshots

Primary key: `league_id + date + team_name (composite PK)`

Columns: position, points, goal_diff, form, played, won, drawn, lost

Notes: Point-in-time league tables. Source: API-Football /standings. Snapshot after each matchday.

transfers

Primary key: `id (PK, auto)`

Columns: player_id, date, type (transfer/loan/free), team_in, team_in_id, team_out, team_out_id

Notes: Career moves. Source: API-Football /transfers.

sidelined

Primary key: `id (PK, auto)`

Columns: player_id, type, start_date, end_date

Notes: Full injury/suspension history. Source: API-Football /sidelined.

injuries

Primary key: `id (PK, auto)`

Columns: player_id, fixture_id, type, reason, date

Notes: Upcoming/recent injury reports. Source: API-Football /injuries.

expected_metrics

Primary key: `player_id + season + source (composite PK)`

Columns: league_id, xg, npxg, xa, xg_per_shot, goals_minus_xg, assists_minus_xa, progressive_passes, progressive_carries, progressive_receptions

Notes: Expected metrics. Source: FBref (canonical) or proxy model (fallback). Source column distinguishes.

market_values

Primary key: `player_id + date (composite PK)`

Columns: market_value_eur, contract_expiry (date, nullable), wage_estimate (nullable)

Notes: Financial data. Source: Transfermarkt. Updated monthly.

player_roles

Primary key: `player_id + season (composite PK)`

Columns: primary_role, secondary_role (nullable), cluster_confidence

Notes: Inferred from k-means clustering. Updated after each season or mid-season refresh.

role_templates

Primary key: `template_id (PK, auto)`

Columns: role_name, version, created_date, created_by, metrics_json (JSONB: {metric_name: weight}), is_active (bool)

Notes: Role template definitions. Versioned. Owned by coaching staff.

briefs

Primary key: `brief_id (PK, auto)`

Columns: role_name, archetype_primary, archetype_secondary (nullable), intent (text), budget_max_fee, budget_max_wage, budget_max_contract_years, age_min, age_max, league_scope (JSONB), timeline, pathway_check_done (bool), pathway_player_id (nullable), status (draft/live/closed), created_date, created_by, approved_by

Notes: Recruitment briefs. Must be fully populated before going live (Section 2.1.1).

pipeline

Primary key: `id (PK, auto)`

Columns: brief_id (FK), player_id (FK), stage (identified/monitoring/longlist/shortlist/scouting/recommended/approved/bid/signed/rejected/archived), archetype_primary, archetype_secondary (nullable), intent (text), added_date, stage_changed_date, stage_changed_by, rejection_reason (nullable)

Notes: Pipeline tracking. One row per player-brief pair.

scout_notes

Primary key: `id (PK, auto)`

Columns: player_id, fixture_id (nullable), scout_name, date, technical_rating (1-5), tactical_rating (1-5), physical_rating (1-5), mental_rating (1-5), system_fit_rating (1-5), notes_text, video_urls (JSONB, nullable)

Notes: Structured scouting observations.

predictions_log

Primary key: `id (PK, auto)`

Columns: player_id, brief_id, prediction_date, model_version, role_fit_score, l1_performance_score, championship_projection_50th, championship_projection_10th, championship_projection_90th, projected_minutes_share, projected_adaptation_months, availability_risk_prob, financial_value_band_low, financial_value_band_high, var_score, composite_score, archetype_weights_used (JSONB)

Notes: Every model prediction logged for validation. Immutable once written.

overrides

Primary key: `id (PK, auto)`

Columns: player_id, brief_id, override_date, overridden_by, original_model_output (JSONB), decision_made, reason_category (tactical/financial/medical/intelligence/alternatives/timing), reason_text, outcome (nullable, filled at post-window audit)

Notes: Override tracking for governance.

outcomes

Primary key: `id (PK, auto)`

Columns: player_id, brief_id, signed_date, fee_paid, wage_annual, contract_years, assessment_date_6mo, assessment_date_12mo, assessment_date_18mo, performance_hit (bool, nullable), financial_hit (bool, nullable), availability_hit (bool, nullable), failure_type (nullable), failure_notes (nullable)

Notes: Outcome tracking for validation. Populated at 6/12/18 month marks.

wyscout_zone_stats

Primary key: `player_id + season + zone (composite PK)`

Columns: zone (own_third/middle_third/final_third/left/central/right), metrics_json (JSONB: all exported metrics), export_date

Notes: Wyscout zone-filtered data for pipeline players. Manual import.

pathway_players

Primary key: `player_id (PK, int)`

Columns: player_name, birth_date, position, primary_role, current_level (academy/dev_squad/loan/first_team), readiness_estimate_months (nullable), last_assessed_date, assessed_by, notes

Notes: Internal pathway players. Maintained by academy staff.

Index Strategy

Create indices on: (player_id, date) on match_performances; (league_id, season) on match_performances; (fixture_id) on match_events, lineups, fixture_team_stats; (player_id) on transfers, sidelined, injuries, expected_metrics, market_values; (brief_id, stage) on pipeline; (player_id, brief_id) on predictions_log, overrides, outcomes.

Component 2: Data Ingestion

2.1 API-Football Ingestion

File: `ingestion/api_football.py`

Core function: `def api_get(endpoint: str, params: dict) -> dict`

Wraps requests.get with API key header, rate limiting (sleep 6s between calls), error handling, and retry logic. Base URL: https://v3.football.api-sports.io

Ingestion functions (one per endpoint):

Function	Endpoint	Target Table	Frequency	Implementation Notes
fetch_match_performances(from_date, to_date)	/fixtures/players	match_performances	Daily	Iterates completed fixtures. Calls /fixtures/players per fixture. Parses ALL fields from stat dict. Inserts with fixture_id + player_id dedup. Stores null for missing stats, NEVER 0.
fetch_fixture_team_stats(from_date, to_date)	/fixtures/statistics	fixture_team_stats	Daily	Called alongside match_performances. One call per fixture returns both teams.
fetch_match_events(from_date, to_date)	/fixtures/events	match_events	Daily	All events per fixture. Store time_elapsed, type, detail, player, assist, comments.
fetch_lineups(from_date, to_date)	/fixtures/lineups	lineups	Daily	Starting XI + bench. Extract formation, grid position, coach. is_starter derived from startXI vs substitutes list.
fetch_standings(league_id, season)	/standings	standings_snapshots	After each matchday	Snapshot with today's date. Used for opposition quality tiering.
fetch_player_profiles(league_id, season)	/players	players	Weekly	Paginated (20/page). Extract bio: birth_date, nationality, height, weight, photo. Season stats used for cross-check only.
fetch_transfers(player_id)	/transfers	transfers	Monthly (or per watchlist player)	Full transfer history. Parse date, type, teams.in, teams.out.
fetch_sidelined(player_id)	/sidelined	sidelined	Monthly (or per	Full absence history. Parse type, start, end dates.

			watchlist player)	
fetch_injuries(player_id_or_league)	/injuries	injuries	Daily (match mornings)	Upcoming/recent injury reports. Parse type, reason, date.

Orchestrator: `ingestion/run_daily_ingest.py`

Calls fetch_match_performances, fetch_fixture_team_stats, fetch_match_events, fetch_lineups for all completed fixtures since last run. Logs fixture count, player count, any errors. Called via cron daily at 06:00.

2.2 FBref Ingestion

File: `ingestion/fbref.py`

Function: `def scrape_fbref_player_stats(league_url: str, season: str) -> pd.DataFrame`

Scrapes FBref player season stats tables using BeautifulSoup. Targets the 'Standard Stats', 'Shooting', 'Passing', and 'Possession' tables. Extracts: xG, npxG, xA, progressive passes, progressive carries, progressive receptions. Maps player names to player_id using fuzzy matching against the players table (name + team + league). Inserts into expected_metrics with source = 'fbref'.

Frequency: Weekly for EFL leagues + top 5 European leagues.

Fallback: If FBref does not cover a league, the proxy xG model (Component 5.1) generates estimates with source = 'proxy'.

2.3 Transfermarkt Ingestion

File: `ingestion/transfermarkt.py`

Function: `def scrape_market_values(league_slug: str, season: str) -> pd.DataFrame`

Scrapes Transfermarkt league squad pages. Extracts: player name, market value (parse £/€ with unit conversion), contract expiry date (where shown). Maps to player_id via fuzzy name + team matching. Inserts into market_values with today's date.

Frequency: Monthly for target leagues. Additionally at the start and end of each transfer window.

2.4 Wyscout Manual Import

File: `ingestion/wyscout_import.py`

Function: `def import_wyscout_export(csv_path: str, player_id: int, season: str, zone: str)`

Reads a Wyscout Advanced Search CSV export. Expects all metrics as columns. Stores the full row as JSONB in wyscout_zone_stats. The zone parameter is set manually when the analyst runs the import (one of: own_third, middle_third, final_third, left, central, right). Timestamps the import.

Trigger: Manual. Run when a player moves from Monitoring to Scouting stage in the pipeline. Analyst exports 6 zone-filtered CSVs from Wyscout and imports each.

Component 3: Feature Engineering

3.1 Per-90 Normalisation

File: `features/per90.py`

Function: `def compute_per90(player_id: int, season: str = None) -> pd.DataFrame`

Loads all match_performances rows for the player (optionally filtered by season). For each volume metric (goals_scored, assists, shots_total, shots_on_target, passes_total, passes_key, tackles_total, tackles_blocks, tackles_interceptions, duels_total, duels_won, dribbles_attempts, dribbles_success, dribbles_past, fouls_committed, fouls_drawn, yellow_cards, saves, pen_won), computes: metric_per90 = (metric / minutes) * 90. Returns a DataFrame with one row per match, original metrics plus all _per90 columns. Matches with minutes < 1 are excluded. Matches with minutes < 20 are flagged as low_minutes = True.

3.2 Rolling Aggregations

File: `features/rolling.py`

Function: `def compute_rolling(player_per90_df: pd.DataFrame) -> dict`

Takes the per90 DataFrame from 3.1. Filters to starts only (minutes >= 45). Computes for EACH per90 metric:

- `roll_3`: mean of last 3 starts
- `roll_5`: mean of last 5 starts
- `roll_10`: mean of last 10 starts
- `season_avg`: mean of all starts in current season
- `roll_10_std`: std deviation over last 10 starts
- `roll_10_cv`: coefficient of variation (std/mean) over last 10 starts
- `trend_slope_10`: linear regression slope over last 10 starts (positive = improving)
- `trend_slope_season`: linear regression slope over all starts in current season

Returns a dict keyed by metric name, each containing the above values. Used downstream by scoring and projection models.

3.3 Opposition Quality Adjustment

File: `features/opposition.py`

Function: `def compute_opposition_splits(player_id: int, season: str) -> dict`

For each match the player appeared in, looks up the opponent's league position from standings_snapshots (using the snapshot closest to but not after the match date). Classifies opponent into Tier 1 (1st–4th), Tier 2 (5th–10th), Tier 3 (11th–16th), Tier 4 (17th+). Then computes the player's per-90 averages for each metric, split by opposition tier. Returns a dict: {metric_name: {tier1: value, tier2: value, tier3: value, tier4: value}}.

3.4 League Intensity Normalisation

File: `features/league_adjust.py`

Function: `def compute_league_percentile(player_id: int, season: str, role: str) -> dict`

For each per-90 metric, computes the player's percentile rank within their league and role group (using player_roles table for role classification). Returns {metric: percentile_0_to_100}. Also computes league-adjusted absolute: (player_per90 / league_role_avg_per90) * reference_league_avg (reference league = Championship, configurable in settings.py).

3.5 Availability Features

File: `features/availability.py`

Function: `def compute_availability_features(player_id: int) -> dict`

Combines match_performances, sidelined, and injuries tables. Computes:

- `availability_rate_season`: starts / total matchdays in current season
- `availability_rate_3yr`: average over last 3 seasons
- `injury_frequency_3yr`: count of sidelined entries in last 3 years
- `avg_injury_duration`: mean days per sidelined entry
- `max_injury_duration`: longest single absence
- `muscle_injury_count`: count where type contains 'muscle', 'hamstring', 'thigh', 'calf', 'groin'
- `recurrence_rate`: same-type injuries / total injuries
- `days_since_last_injury`: days since most recent sidelined.end_date
- `minutes_continuity`: consecutive matches with >= 60 minutes (current run)
- `subbed_off_rate`: % of starts where subbed before 80th min (from match_events sub events)

Returns a flat dict of all features.

3.6 Career Trajectory Features

File: `features/trajectory.py`

Function: `def compute_trajectory_features(player_id: int) -> dict`

Uses transfers table + player bio + match_performances. Computes: league_level_history (list of league tiers by season), loan_count, clubs_count, countries_played_in, age_at_first_senior_appearance, seasons_at_current_club, output_trajectory_2yr (slope of key per-90 metrics across last 2 seasons), age_curve_position (pre-peak/peak/post-peak based on position-specific curves from config/age_curves.json).

3.7 Role Classification

File: `features/role_classification.py`

Function: `def classify_roles(season: str, position_group: str) -> pd.DataFrame`

Loads all players in the given position group (G/D/M/F) for the season. Builds a feature matrix of league-adjusted per-90 metrics (the 10–12 most discriminating for that position group). Runs k-means with k=4–6 (determined by silhouette score). Maps each cluster to a named role by comparing cluster centroids against the role_templates definitions. Assigns primary_role and secondary_role (if player is close to two cluster boundaries). Writes to player_roles table.

Important: Clustering output is validated by coaching staff before being used in scoring. If clusters do not align with football reality, adjust k or feature selection. Clustering is an aid, not the truth.

3.8 Sample Size Confidence

File: `features/confidence.py`

Function: `def compute_confidence(player_id: int, season: str) -> dict`

Counts tracked appearances (minutes > 0). Returns:

- `appearances`: raw count
- `confidence_tier`: Low (<10), Medium (10-20), High (20-30), VeryHigh (30+)
- `shrinkage_factor`: appearances / (appearances + 10). Approaches 1.0 as sample grows.

When downstream models use per-90 values for Low confidence players, they should compute: shrunk_value = (player_value * shrinkage_factor) + (league_role_avg * (1 - shrinkage_factor))

3.9 GBE Eligibility Estimate

File: `features/gbe.py`

Function: `def estimate_gbe_score(player_id: int) -> dict`

Loads player nationality, appearances in continental competition, domestic league tier, and international cap status (approximated from match data in international competitions if tracked). Computes estimated GBE points per FA published criteria. Returns: {points_estimate: int, status: green/amber/red, gaps: [list of missing criteria], notes: str}. Green = likely qualifies. Amber = borderline. Red = unlikely. Applies to non-UK/Irish players only; UK/Irish players always return green.

Component 4: Hard Gate Filtering Engine

File: `gates/filtering.py`

Function: `def apply_gates(player_id: int, brief: dict) -> dict`

Takes a player_id and a brief dict (from briefs table). Applies each gate in order. Returns:

- `{passed: bool, gate_results: [{gate_name, class, result: pass/fail/caution/exception, detail: str}]}`

Gate application order (cheapest first):

1. **Registration eligibility** (Absolute): check nationality against EFL registration rules. If international, check GBE status.
2. **Affordability** (Absolute): estimated market value + (estimated wage * brief.budget_max_contract_years) <= brief.budget_max_fee + (brief.budget_max_wage * brief.budget_max_contract_years).
3. **Role relevance** (Absolute): player's primary_role or secondary_role must match the brief's role_name.
4. **Pathway blocking (hard)** (Absolute): if brief.pathway_player_id is set AND pathway player's readiness < 12 months AND external player age <= pathway player age AND brief contract > 2 years.
5. **Wage discipline** (Caution): if estimated wage > 1.2 * club wage band for role.
6. **Age-profile fit** (Caution): check age against archetype range AND position-specific curve from config/age_curves.json.
7. **Availability risk** (Caution): if availability_risk_prob (from Model 3) > 0.40.
8. **Data confidence** (Caution): if confidence_tier == Low.
9. **Championship projection** (Exception): if archetype is Championship Transition or Emerging Asset AND championship_projection_50th < 40th percentile.
10. **Pathway blocking (soft)** (Exception): if pathway player exists at role with readiness 12–24 months AND external contract > 2 years.

Batch function: `def filter_universe(brief: dict) -> pd.DataFrame`

Loads all players from tracked leagues matching the brief's league scope. Applies apply_gates to each. Returns a DataFrame of players who passed all Absolute gates, with Caution and Exception flags attached as columns. This is the market scan output (Stage 2 of the decision framework).

Component 5: Model Stack

5.1 Proxy xG Model (Fallback)

File: `models/proxy_xg.py`

Function: `def train_proxy_xg(statsbomb_data_path: str) -> sklearn.Pipeline`

Trains a logistic regression on StatsBomb open data shot events. Features: is_header (bool), angle_to_goal (derived), distance_to_goal (derived), is_penalty (bool), is_direct_free_kick (bool), game_state (winning/drawing/losing). Target: goal (bool). Saves trained model to disk.

Function: `def predict_xg(player_id: int, season: str) -> float`

For leagues without FBref coverage: loads the player's match_events goal events, extracts available features, predicts per-shot xG, sums to season xG estimate. Writes to expected_metrics with source = 'proxy'.

5.2 Role-Fit Model

File: `models/role_fit.py`

Function: `def score_role_fit(player_id: int, template_id: int, season: str) -> dict`

Loads the role template (from role_templates table, must be is_active=True). Loads the player's league-adjusted per-90 percentiles (from features/league_adjust.py). For each metric in the template, computes: player_percentile * template_weight. Sums to produce raw_role_fit_score (0–100). Also returns per-metric decomposition: {metric_name: {player_percentile, weight, contribution}}.

Model type: Weighted similarity (not ML). Deterministic and fully explainable. The weights come from the role template, which is owned by coaching staff.

Returns: `{score: float, decomposition: dict, template_version: str, confidence_tier: str}`

5.3 League One Performance Model

File: `models/l1_performance.py`

Function: `def score_l1_performance(player_id: int, season: str, role: str) -> dict`

Loads the player's league-adjusted per-90 percentiles within their current league for the given role. Also loads: consistency score (1/CV from rolling.py), form trend (trend_slope_10), output vs. Tier 1 opposition (from opposition.py). Combines into a current performance composite using fixed weights: 50% league-adjusted per-90 average across role-defining metrics, 20% consistency score, 15% form trend (normalised to 0–100), 15% Tier 1 output percentile.

Returns: `{score: float, percentile_in_role: float, form_trend: str (improving/stable/declining), consistency: float, vs_tier1_percentile: float}`

5.4 Championship Projection Model

File: `models/championship_projection.py`

Training function: `def train_projection_model(training_data: pd.DataFrame) -> dict`

Training data: historical players who moved from L1 (or comparable leagues) to Championship with >= 15 appearances in both. For each player, computes features: origin per-90 metrics (season avg), origin league intensity profile, destination league intensity profile, age at transfer, primary role, origin team league position (proxy for team strength), destination team league position. Target: destination per-90 metrics (first full season, per metric). Model: statsmodels mixed-effects regression (hierarchical) with random intercepts per league-pair. Produces estimates with prediction intervals. Saves model per metric.

Prediction function: `def project_to_championship(player_id: int, season: str) -> dict`

Loads player's current features. Applies trained models per role-defining metric. Returns:

- `projected_performance`: {metric: {p10, p50, p90}} for each role-defining metric
- `projected_minutes_share`: probability of being a starter (>=25 starts) at Championship. From a separate logistic regression trained on same transition data. Returns probability 0–1.0.
- `projected_adaptation_months`: estimated months before reaching steady-state. Computed from historical first-season vs. second-season output ratios for movers in the training data. Returns months estimate.
- `sample_size`: number of historical cases in this league-pair used for training
- `confidence_note`: if sample_size < 20, return 'Low sample — wide uncertainty bands, proceed with caution'

5.5 Availability Risk Model

File: `models/availability_risk.py`

Training function: `def train_availability_model(training_data: pd.DataFrame) -> sklearn.Pipeline`

Training data: all players in database with >= 2 seasons of history. Features: the output of features/availability.py (3yr rate, injury frequency, muscle count, recurrence, max duration, age, position). Target: binary — was the player available for >= 75% of matchdays in the following season? Model: LogisticRegression (interpretable). Publish coefficients.

Prediction function: `def predict_availability_risk(player_id: int) -> dict`

Returns: {probability_available_75pct: float, risk_tier: Low/Medium/High, contributing_factors: [{factor, direction, magnitude}], caveat: 'Advisory to medical. Does not predict freak injuries.'}

5.6 Financial Value Model

File: `models/financial_value.py`

Training function: `def train_value_model(training_data: pd.DataFrame) -> sklearn.Pipeline`

Training data: historical transfers with known fees (from Transfermarkt) + player features at time of transfer (age, position, role, per-90 output, league level, contract remaining, market value pre-transfer). Model: sklearn.QuantileRegressor for quantiles 0.1, 0.5, 0.9. Produces value bands.

Prediction function: `def estimate_value(player_id: int, brief: dict) -> dict`

Returns:

- `fair_value_band`: {low (10th pct), mid (50th pct), high (90th pct)} in GBP
- `wage_fit`: proposed wage vs. club band for role. Returns ok/caution/exceeds.
- `resale_band_2yr`: projected market value in 2 years (using age curve + output trajectory)
- `replacement_cost`: estimated cost to replace this player's output from the market if they leave
- `var_score`: composite quality score / normalised total cost. Above 1.0 = good value.
- `comparable_transactions`: list of 3–5 similar recent transfers with fees

5.7 Similarity & Alternatives Engine

File: `models/similarity.py`

Function: `def find_similar(player_id: int, role: str, n: int = 10, filters: dict = None) -> list`

Loads target player's league-adjusted per-90 vector for role-defining metrics. Loads all other players at the same role. Computes cosine distance. Optionally filters by age range, league, budget, confidence tier. Returns top n most similar players with: player_id, name, similarity_score, per-metric comparison (where they match and differ), and any hard gate flags.

Component 6: Composite Scoring Engine

File: `scoring/composite.py`

Function: `def compute_composite(player_id: int, brief: dict, season: str) -> dict`

This is the core scoring function. It orchestrates all upstream components and applies the mechanics from Spec Section 5.2.

Steps:

11. Load brief: archetype_primary, archetype_secondary, role template, budget
12. Load archetype weights from config/archetype_weights.json for primary and secondary
13. Blend weights: blended = 0.70 * primary_weights + 0.30 * secondary_weights (if secondary exists, else 100% primary)
14. Compute Layer 1 — Tactical Fit: for now, use role_fit_score as proxy (tactical fit refinement is Phase 2 with Wyscout zone data and pressing metrics). Score 0–100.
15. Compute Layer 2 — Role Fit: call models/role_fit.py. Score 0–100.
16. Compute Layer 3 — Current Performance: call models/l1_performance.py. Score 0–100.
17. Compute Layer 4 — Upward Projection: call models/championship_projection.py. Convert 50th percentile projection to 0–100 scale relative to Championship role distribution.
18. Compute Layer 5 — Financial Value: call models/financial_value.py. Convert var_score to 0–100 scale (VAR 1.0 = 50, scaled linearly).
19. Compute Layer 6 — Availability: call models/availability_risk.py. Convert probability_available_75pct to a 0–1.0 multiplier.

Mechanics:

- **Tactical fit cap:** `if tactical_fit_score < 40: composite = min(composite, 60)`
- **Weighted sum:** `raw_composite = sum(layer_score * blended_weight for each layer except availability)`
- **Availability discount:** `composite = raw_composite * availability_multiplier`
- **Projection override for Emerging Asset:** `if primary_archetype == 'emerging_asset' and projection_50th_pct >= 70: allow current_performance floor of 30 (instead of default 50)`

Returns:

- `{composite_score, per_layer_scores: {tactical_fit, role_fit, current_performance, projection, financial, availability}, per_layer_weights_used, archetype_primary, archetype_secondary, tactical_fit_cap_applied: bool, confidence_tier, projection_band: {p10, p50, p90}, internal_pathway_comparison: {pathway_player_id, pathway_gap_pct} or null}`

Logging: Every call to compute_composite writes a row to predictions_log table with all inputs and outputs. This is immutable and used for validation.

Component 7: Pipeline & Workflow Management

File: `governance/pipeline.py`

Function: `def create_brief(params: dict) -> int`

Validates that ALL required fields are present (Section 2.1.1 of spec). If any missing, raises ValueError with specific missing field. If pathway_check_done is False, raises ValueError('Pathway check required before brief goes live'). Inserts into briefs table with status='live'. Returns brief_id.

Function: `def generate_longlist(brief_id: int) -> pd.DataFrame`

Calls gates/filtering.py filter_universe. Then scores all passing players with scoring/composite.py. Ranks by composite_score. Returns top 60. Logs all predictions.

Function: `def promote_to_shortlist(brief_id: int, player_id: int) -> bool`

Checks ALL conditions from Spec Section 2.1.3: role_fit >= 60th pct, confidence >= Medium, Championship projection >= 40th pct (for relevant archetypes), financial assessment completed, pathway comparison done. If all met, updates pipeline stage to 'shortlist'. If not, returns False with list of unmet conditions.

Function: `def check_scouting_requirements(brief_id: int, player_id: int) -> dict`

Checks Section 2.1.2: counts scout_notes rows for this player (need >= 3). Checks if any note has opposition quality 'top-6'. Checks if coach system_fit sign-off exists. Checks if Wyscout zone data exists (for Championship Transition / Emerging Asset archetypes). Returns {ready: bool, missing: [list]}.

Function: `def log_override(player_id: int, brief_id: int, original_output: dict, decision: str, reason_category: str, reason_text: str, overridden_by: str)`

Writes to overrides table. Validates reason_category is one of: tactical, financial, medical, intelligence, alternatives, timing.

Component 8: Output Generation

8.1 Ranked Longlist

File: `outputs/longlist.py`

Function: `def generate_longlist_report(brief_id: int) -> str`

Loads longlist from pipeline table. For each player: composite score, per-layer breakdown, archetype, confidence tier, projection band (p10–p90), top 3 strengths, top 2 concerns, pathway comparison flag. Renders to HTML using templates/longlist.html Jinja2 template. Can be exported to PDF.

8.2 Shortlist Card

File: `outputs/shortlist_card.py`

Function: `def generate_shortlist_card(brief_id: int, player_id: int) -> str`

Deep per-player profile: all 6 layers with full decomposition. Role-fit radar chart data (per-metric). Form trajectory (last 10 matches). Opposition splits. Availability timeline. Financial summary (fee band, wage fit, VAR, comparables). Projection bands. Career trajectory. Scout notes summary. Pathway comparison. Hard gate status. Renders to HTML/PDF.

8.3 Target Comparison Sheet

File: `outputs/comparison.py`

Function: `def generate_comparison(brief_id: int, player_ids: list) -> str`

Side-by-side for 2–4 players. Per-layer scores aligned in columns. Highlights where candidates differ most (> 15 percentile points). Shows projection ranges overlaid. Financial comparison row. Availability comparison row. Notes which differences are within confidence intervals (not meaningful). Renders to HTML/PDF.

8.4 Recommendation Pack

File: `outputs/recommendation.py`

Function: `def generate_recommendation_pack(brief_id: int, recommended_player_id: int, alternatives: list) -> str`

Board-ready document. Sections: executive summary (1 paragraph), player profile, 6-layer breakdown, projection with ranges, financial impact (fee, wages, TCO, resale projection, replacement cost), availability risk with medical sign-off status, scout report summary, pathway impact assessment, alternatives (2–3 other shortlisted players as fallbacks), confidence disclosure (what we know well, what is uncertain, what is scouting-only), override history (any overrides that occurred during the process). Renders to HTML/PDF.

Component 9: Validation Framework

File: `models/validation.py`

Function: `def temporal_backtest(model_name: str, window_dates: list) -> dict`

For each window date: retrain the specified model using only data before that date. Generate predictions for transfers in that window. Compare against actual outcomes (from outcomes table, or historical performance data). Return per-window metrics: AUC, precision@20, calibration curve, coverage of prediction intervals.

Function: `def compute_outcome_metrics(window: str = None) -> dict`

Loads outcomes table. For each signed player with sufficient assessment data: classifies as hit/miss per Spec Section 13.1 definitions (performance hit: >= 50th pct for 2 consecutive 6-month periods; financial hit: value >= 100% of acquisition; availability hit: >= 75% matchdays). Returns summary: total signed, total assessed, hit rates by type, model-added wins, missed opportunities identified.

Function: `def post_window_audit(window: str) -> dict`

Comprehensive audit: briefs created, longlists generated, shortlists produced, players progressed by stage, signed, overrides logged, override outcomes (where available), data quality incidents, coverage gaps discovered. Returns structured dict for report generation.

Function: `def calibration_check() -> dict`

For all predictions in predictions_log with observable outcomes: bin by predicted probability (deciles). Compare predicted vs. actual success rate per bin. Flag if any bin deviates by > 5 percentage points. Return calibration curve data and drift alert (bool).

Component 10: Build Order

Build components in this exact order. Each depends on the ones before it.

Order	Component	Files	Depends On	Test Criteria
0	Project structure + config	Directory tree + all config/*.json + config/settings.py	Nothing	All config files load without error. DB connection test passes.
1	Database schema	db/schema.py + migration	Component 0	All tables created. Can insert and query a test row in each.
2a	API-Football ingestion	ingestion/api_football.py	Component 1	Fetch 1 fixture. Verify all fields populated in match_performances. Null stored as null.
2b	FBref ingestion	ingestion/fbref.py	Component 1	Scrape 1 league. Verify xG/xA values match FBref website. Player name matching works.
2c	Transfermarkt ingestion	ingestion/transfermarkt.py	Component 1	Scrape 1 league. Verify market values parse correctly. Currency conversion works.
2d	Wyscout import	ingestion/wyscout_import.py	Component 1	Import 1 test CSV. Verify JSONB storage and zone tagging.
2e	Daily orchestrator	ingestion/run_daily_ingest.py	Components 2a–2d	Full daily run completes without error. Dedup works. Logs fixture count.
3a	Per-90 normalisation	features/per90.py	Component 2a	Pick 3 known players. Verify per-90 values match manual calculation.
3b	Rolling aggregations	features/rolling.py	Component 3a	Verify rolling means, std, CV, and trend slopes for a known player.
3c	Opposition quality splits	features/opposition.py	Components 2a, 1 (standings)	Verify tier classification for 5 known matches. Verify per-tier averages.
3d	League intensity normalisation	features/league_adjust.py	Components 3a, role classification	Verify percentile for a known player matches manual calculation.
3e	Availability features	features/availability.py	Components 2a, 1 (sidelined/injuries)	Verify availability rate and injury counts for 3 known players.
3f	Career trajectory	features/trajectory.py	Components 2a, 1 (transfers, players)	Verify league level history and age curve position for known players.
3g	Role classification	features/role_classification.py	Component 3d	Run clustering. Review clusters with football knowledge. Validate against known player roles.

3h	Confidence tiers	features/confidence.py	Component 2a	Verify appearance counts and tier assignments.
3i	GBE estimation	features/gbe.py	Components 1 (players, match_performances)	Verify GBE score for 3 known non-UK players against FA criteria.
4	Hard gate filtering	gates/filtering.py	All Component 3	Create a test brief. Run filtering. Verify known-ineligible players are excluded. Verify caution flags attach correctly.
5a	Proxy xG model	models/proxy_xg.py	StatsBomb open data (external)	Train on open data. Verify calibration curve. Cross-validate.
5b	Role-Fit model	models/role_fit.py	Components 3d, 1 (role_templates)	Score 5 known players. Verify decomposition is sensible.
5c	L1 Performance model	models/l1_performance.py	Components 3a–3d	Score 5 known players. Verify ranking aligns with football knowledge.
5d	Championship Projection	models/championship_projection.py	Components 3a–3f + historical transfer data	Train on historical L1→Champ movers. Run temporal backtest on 1 held-out window. Check coverage.
5e	Availability Risk	models/availability_risk.py	Component 3e	Train on historical data. Publish coefficients. Check AUC > 0.65 on held-out season.
5f	Financial Value	models/financial_value.py	Components 2c, 3a, 3f	Train on historical transfers with known fees. Check value band coverage on held-out window.
5g	Similarity engine	models/similarity.py	Components 3d, 3g	Find similar players for 3 known targets. Review results for football sense.
6	Composite scoring	scoring/composite.py	All Component 5	Score 10 known players across 2 archetypes. Verify weight blending. Verify tactical fit cap. Verify availability discount.
7	Pipeline & workflow	governance/pipeline.py	Component 6	Create brief (test validation). Generate longlist. Promote to shortlist (test conditions). Log override.
8a	Longlist output	outputs/longlist.py + templates/	Component 7	Generate longlist report. Visual review: does it look right?
8b	Shortlist card	outputs/shortlist_card.py + templates/	Component 7	Generate card for 1 player. Visual review.
8c	Comparison sheet	outputs/comparison.py + templates/	Component 7	Generate comparison for 3 players. Visual review.
8d	Recommendation pack	outputs/recommendation.py + templates/	Component 7	Generate full pack. Visual review by HoR.

9	Validation framework	models/validation.py	All above	Run temporal backtest on held-out windows. Generate post-window audit for most recent window.

LLM Prompting Strategy

Feed each component to the LLM one at a time in this order. Include: (1) this component's specification from this document, (2) the database schema it depends on, (3) any upstream function signatures it calls, and (4) the test criteria. Review each component's output before moving to the next. Do not batch. Each component should be a separate conversation or prompt.

Total estimated build time: 12–16 weeks for one analyst-developer following this guide. Components 0–4 take ~4 weeks. Components 5–6 take ~4 weeks. Components 7–9 take ~4 weeks. The first usable output (longlist generation) is available at week 10–12.

This document is the bridge between the recruitment model specification and working code. Every function signature, every table definition, and every build dependency is defined. An LLM given this document and the spec can generate each component file-by-file in the correct order.

Document version: 1.0 | March 2026 | Confidential